==

Q1. Student Information

Create a student object containing:

- id
- name
- age
- course
- city

Write a function displayStudent(student).

Use object destructuring inside the function to print all details.

Q2. Product Discount Calculator

Create a product object containing:

- name
- price
- discount

Write a function calculateFinalPrice(product).

Calculate:
- Discount Amount
- Final Price

Use object destructuring inside the function.

Q3. Employee List

Create an array of employee objects.

Each employee contains:

- name
- salary

Using a loop, print:

Employee Name : ______
Salary : ______

Q4. Number Checker

Create an object containing:

- number

Write a function checkNumber(obj).

Use object destructuring inside the function.

Print whether the number is:

- Even
- Odd

Q5. Find a Student

Create an array of student objects.

Each object contains:

- id
- name
- age

Take an ID from the user.

Use the find() method to search for the student.

If found, print all details.

Otherwise print:

Student Not Found

==

Q6. Highest Marks

Create an array of student objects.

Each object contains:

- name
- marks

Using a loop,

find the student who scored the highest marks.

Use object destructuring only while printing the topper.

Q7. Count Adults

Create an array of person objects.

Each object contains:

- name
- age

Use filter() to create a new array containing only adults (age >= 18).

Print:

- Adult count
- Adult names

Q8. Shopping Cart

Create an array of product objects.

Each object contains:

- name
- price
- quantity

Use reduce() to calculate the total bill.

Create a function calculateBill(products).

Print the final amount.

Q9. Search Product

Create an array of product objects.

Each product contains:

- name
- price
- category

Take a product name from the user.

Use find() to search for the product.

If found,

use object destructuring to print:

- Name
- Price
- Category

Otherwise print:

Product Not Found

Q10. Student Report

Create an array of student objects.

Each object contains:

- name
- marks

Use map() to create a new array containing:

```
{
    name,
    grade
}
```

Grade Rules

90+ -> A
75+ -> B
60+ -> C
Below 60 -> D

Print the new array.

===

Q11. Department Salary Report

Create an array of employee objects.

Each employee contains:

- id
- name
- department
- salary

Write a function:

departmentSalary(employees)

Use reduce() to calculate the total salary paid in each department.

Example Output

IT : 120000
HR : 75000
Sales : 90000

Q12. Student Result Analysis

Create an array of student objects.

Each student contains:

- name
- marks (array of 3 subjects)

Write a function analyzeStudents(students).

For every student:

- Calculate Total
- Calculate Average
- Print Pass/Fail

Use array destructuring for marks.

Use object destructuring wherever appropriate.

Q13. Library Management

Create an array of book objects.

Each book contains:

- title
- author
- price
- available

Tasks

1. Use filter() to display only available books.

2. Use map() to print only:

- Title
- Author

Use object destructuring inside map().

Q14. Product Inventory

Create an array of product objects.

Each product contains:

- id
- name
- stock
- price

Write separate functions to:

1. Display all products.

2. Find a product using find().

3. Check if every product is in stock using every().

4. Check if some products are out of stock using some().

5. Calculate total inventory value using reduce().

Use object destructuring only while printing product details.

Q15. Student Management Report

Create an array of student objects.

Each student contains:

- id
- name
- city
- course
- marks (array)

Write separate functions:

1. displayStudents()

2. findTopper()

3. searchStudent(id)

4. calculateAverage()

5. passedStudents()

Requirements

• Use map() to create a report card.
• Use filter() to display passed students.
• Use reduce() to calculate the class average.
• Use find() to search by ID.
• Use object destructuring whenever printing student details.
• Use array destructuring while accessing marks.

==
BONUS CHALLENGE
==

Create a complete Student Management System.

Each student object contains:

- id
- name
- age
- city
- course
- marks (array of 3 subjects)

Write separate functions for:

1. Display all students.
2. Search student by ID (find()).
3. Search students by city (filter()).
4. Display students by course (filter()).
5. Find the topper.
6. Find the lowest scorer.
7. Calculate total marks of every student (map()).
8. Calculate average marks of every student (map()).
9. Display only passed students (filter()).
10. Display only failed students (filter()).
11. Count total passed students.
12. Count total failed students.
13. Check if every student has passed (every()).
14. Check if at least one student scored above 95 (some()).
15. Calculate class average using reduce().
16. Calculate highest marks.
17. Calculate lowest marks.
18. Sort students by name.
19. Sort students by average marks.
20. Print a complete report card for every student.

Use:
✓ Objects
✓ Object Destructuring
✓ Array Destructuring
✓ Functions

✓ Loops
✓ map()
✓ filter()
✓ reduce()
✓ find()
✓ some()
✓ every()

wherever they are most appropriate—not in every question.

==
END OF PRACTICE SET
==